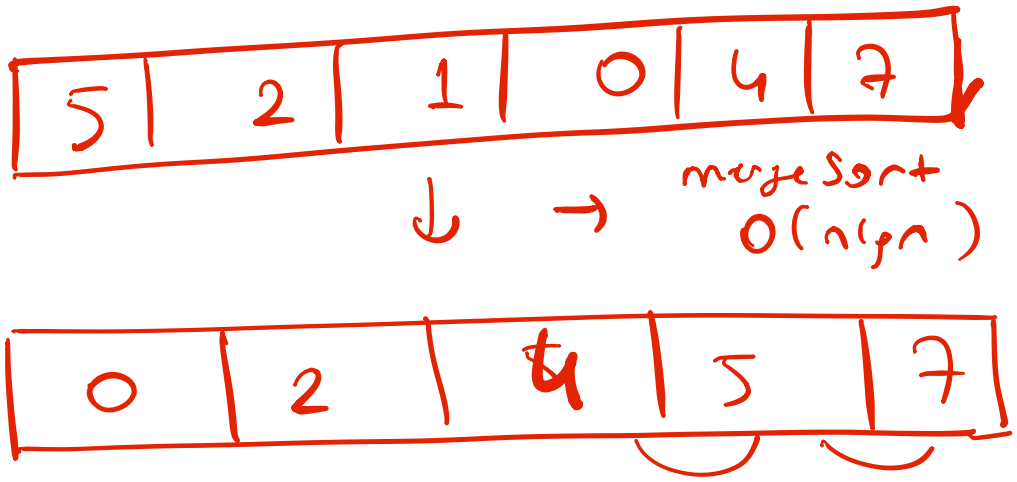
6.1

1. Consider the problem of finding the distance between the two closest numbers in an array of n numbers. (The distance between two numbers x and y is computed as $|x - y|$.)
   a. Design a presorting-based algorithm for solving this problem and determine its efficiency class.
   b. Compare the efficiency of this algorithm with that of the brute-force algorithm (see Problem 9 in Exercises 1.2).

a)



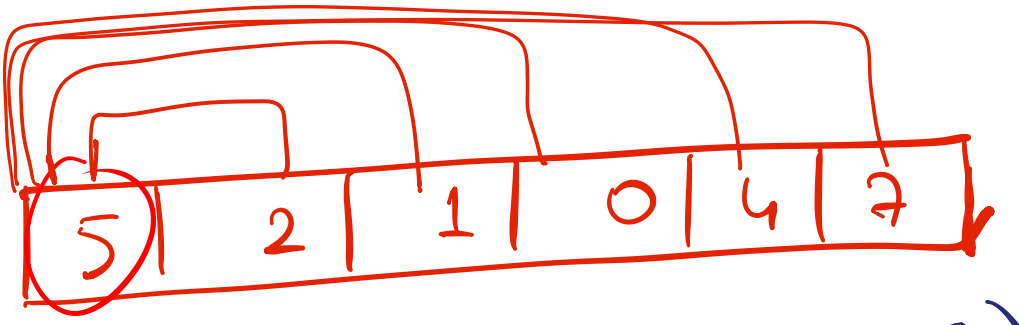
→ Sort the given array → O(nlgn)

→ Compute distance to find the smallest consecutive elements → Θ(n)

→ Overall Complexity: O(nlgn) + Θ(n)

= O(nlgn)

b)



Algorithm Brute Force ($A[0 \ldots n-1]$)

$d \leftarrow \infty$

for $i := 0$ to $n-1$

for $j := 0$ to $n-1$

if $i \neq j$ and $|A[i]-A[j]| < d$

$d \leftarrow |A[i]-A[j]|$

return d

$$\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 \Rightarrow \Theta(n^2)$$

6.1

3. Consider the problem of finding the smallest and largest elements in an array of n numbers.

a. Design a presorting-based algorithm for solving this problem and determine its efficiency class.

b. Compare the efficiency of the three algorithms: (i) the brute-force algorithm, (ii) this presorting-based algorithm, and (iii) the divide-and-conquer algorithm (see Problem 2 in Exercises 5.1).

a) → $O(n \log n)$

- Sort the elements of array
- Return first and last elements of the array
  ↳ $\Theta(1)$

Total complexity = $O(n \log n) + \Theta(1)$

$= O(n \log n)$

b) Brute Force (A[0 ... n-1])

```
max ← -∞
min ← ∞
for i=0 to n-1
    if A[i] > max
        max ← A[i]
    if A[i] < min
        min ← A[i]
return min, max
```

$\Theta(n)$

Algorithm DC ($A[0 \ldots n-1], l, r$)

If A has only one elmts

$min \leftarrow A[l]$

$max \leftarrow A[l]$

else if A has two elmts

If $A[l] > A[r]$

$max \leftarrow A[l]$

$min \leftarrow A[r]$

else $max \leftarrow A[r]$

$min \leftarrow A[l]$

else //

$max, min \leftarrow DC[A[0, \ldots \lfloor \frac{l+r}{2} \rfloor], l, r]$

$max_2, min_2 \leftarrow DC([A[\lfloor \frac{l+r}{2} \rfloor + 1, \ldots r], l, r]$

If $max_2 > max$

$max \leftarrow max_2$

If $min_2 < min$

$min \leftarrow min_2$

return max, min

$$c(n) = 2c(n/2) + 2 \quad \text{for } n > 2$$

$$c(1) = 0$$

$$c(2) = 1$$

$$n = 2^k$$

$$c(2^k) = 2c(2^{k-1}) + 2$$

$$c(2^{k-1}) = 2c(2^{k-2}) + 2$$

$$= 2[2 \cdot c(2^{k-2}) + 2] + 2$$

$$= 2^2 \cdot c(2^{k-2}) + 2^2 + 2$$

$$\vdots$$

$$= 2^i \cdot c(2^{k-i}) + 2^i + 2^{i-1} + \cdots + 2$$

$$\vdots$$

$$= 2^{k-1} \cdot c(2^{k-k+1}) + 2^{k-1} + 2^{k-2} + \cdots + 2$$

$$= 2^{k-1} \cdot \underbrace{c(2)}_{1} + 2^{k-1} + 2^{k-2} + \cdots + 2$$

$$= 2^{k-1} + 2^{k-1} + 2^{k-2} + \cdots + 2$$

$$= 2^{k-1} + 2\left[\frac{2^{k-2} + 2^{k-1} + \cdots + 1}{\frac{2^{k-2+1} - 1}{2-1}}\right]$$

$$= 2^{k-1} + 2(2^{k-1} - 1)$$

$$= 2^{k-1} + 2 \cdot 2^{k-1} - 2$$

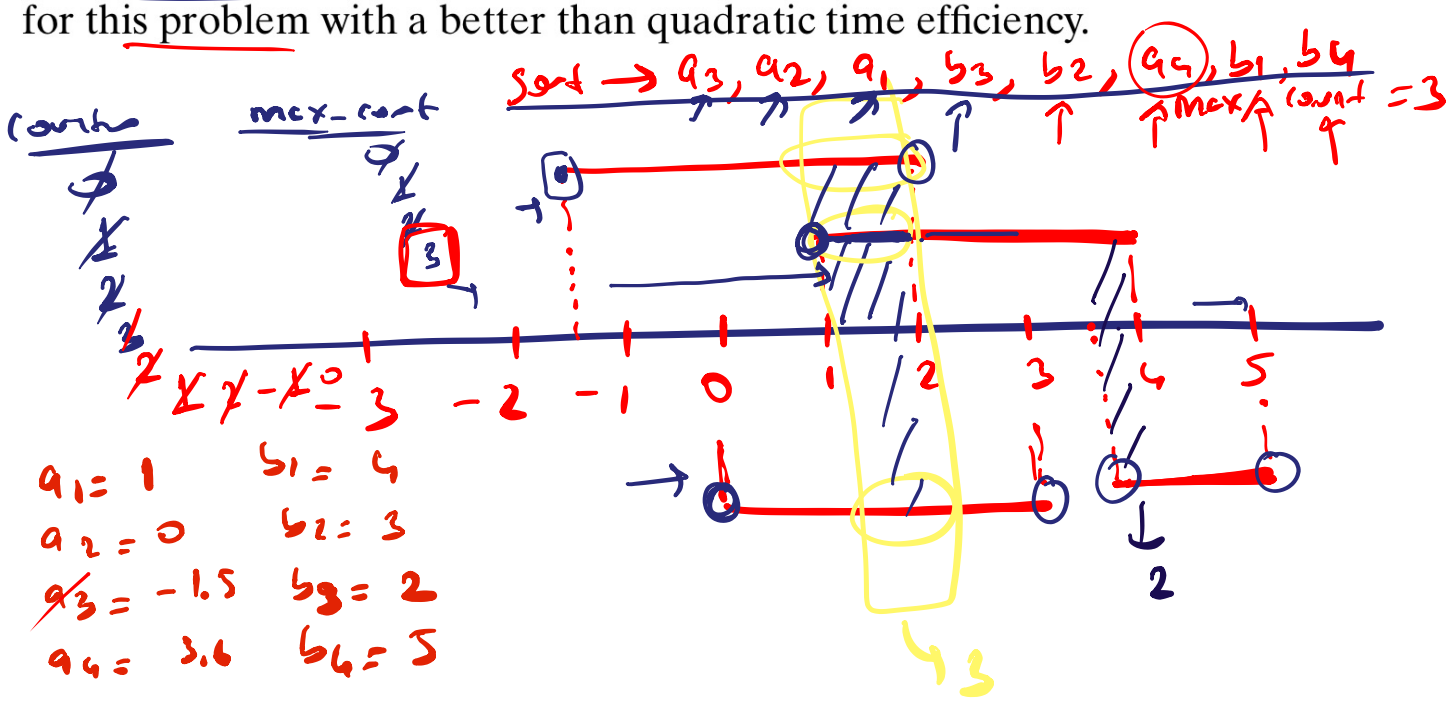
$$= 3 \cdot 2^{k-1} - 2$$

$$= 3 \cdot \frac{2^k}{2} - 2 \qquad (k \to n)$$

$$= \frac{3n}{2} - 2 \to O(n)$$

6.1

8. You have a list of n open intervals $(a_1, b_1), (a_2, b_2), \ldots, (a_n, b_n)$ on the real line. (An open interval (a, b) comprises all the points strictly between its endpoints a and b, i.e., $(a, b) = \{x | \ a < x < b\}$.) Find the maximum number of these intervals that have a common point. For example, for the intervals $(1, 4)$, $(0, 3)$, $(-1.5, 2)$, $(3.6, 5)$, this maximum number is 3. Design an algorithm for this problem with a better than quadratic time efficiency.



→ Sort all data points $(a_i, b_j, \ldots)$

If $a_i == b_j$, assume $b_j < a_i$

→ Scan the sorted list from left to right

counter ← 0, max_overlap ← 0

- Increase counter by 1 if we encounter a starting point (a_i)
- Decrease the counter by 1 if we encounter an ending point (b_i)
- Update max_count if counter > max_overlap

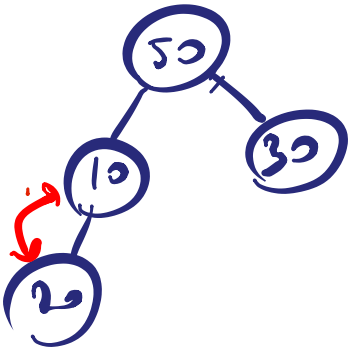
return max_overlap

6.4

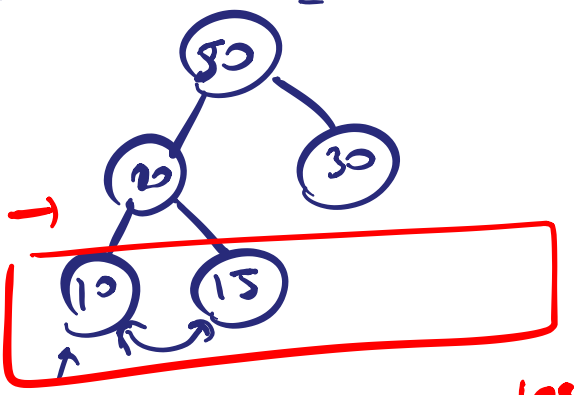
5. a. Design an efficient algorithm for finding and deleting an element of the smallest value in a heap and determine its time efficiency.

b. Design an efficient algorithm for finding and deleting an element of a given value v in a heap H and determine its time efficiency.

50, 10, 30, 20, 15 → max-heap

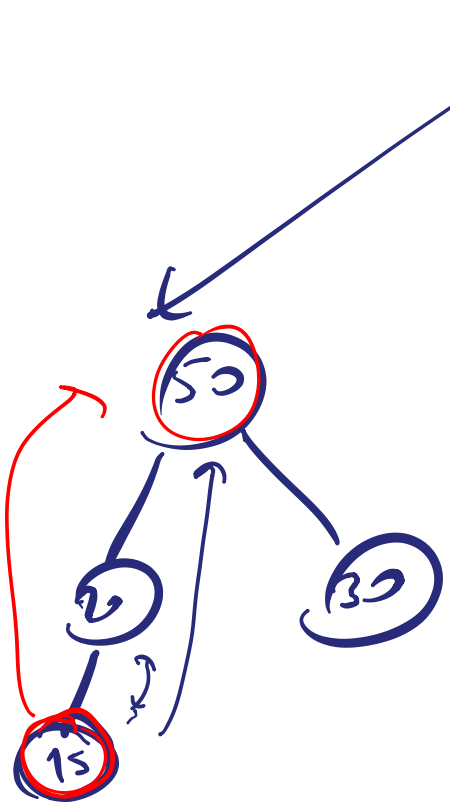


→



leaf nodes

- Search leaf nodes → O(n)
- To delete an element in a max heap
  - Replace A[n] and A[1]
  - check the heap condition
  - Heapify the graph if necessary → O(log n)



deletion

→ Overall complexity = O(n) + O(1) + O(log n)

= O(n)

a) – Find the value k in a heap → $O(n)$

– After finding the value k in position i.

To delete it:

- $\Theta(1)$ exchange $H[i]$ and $H[n]$
- $\Theta(1)$ Decrease heap's size by 1
- $O(\lg n)$ Heapify the structure

→ Deletion operation for an arbitrary key after the finding:

$$\Theta(1) + \Theta(1) + O(\lg n)$$

$$= O(\lg n)$$

Overall Complexity = Finding + Deleting

$$= O(n) + O(\lg n) = O(n)$$

6.6 → Reduction

2. You are given a list of numbers for which you need to construct a min-heap. (A min-heap is a complete binary tree in which every key is less than or equal to the keys in its children.) How would you use an algorithm for constructing a max-heap (a heap as defined in Section 6.4) to construct a min-heap?

Our Aim: Construct a min-heap using a max-heap algorithm.

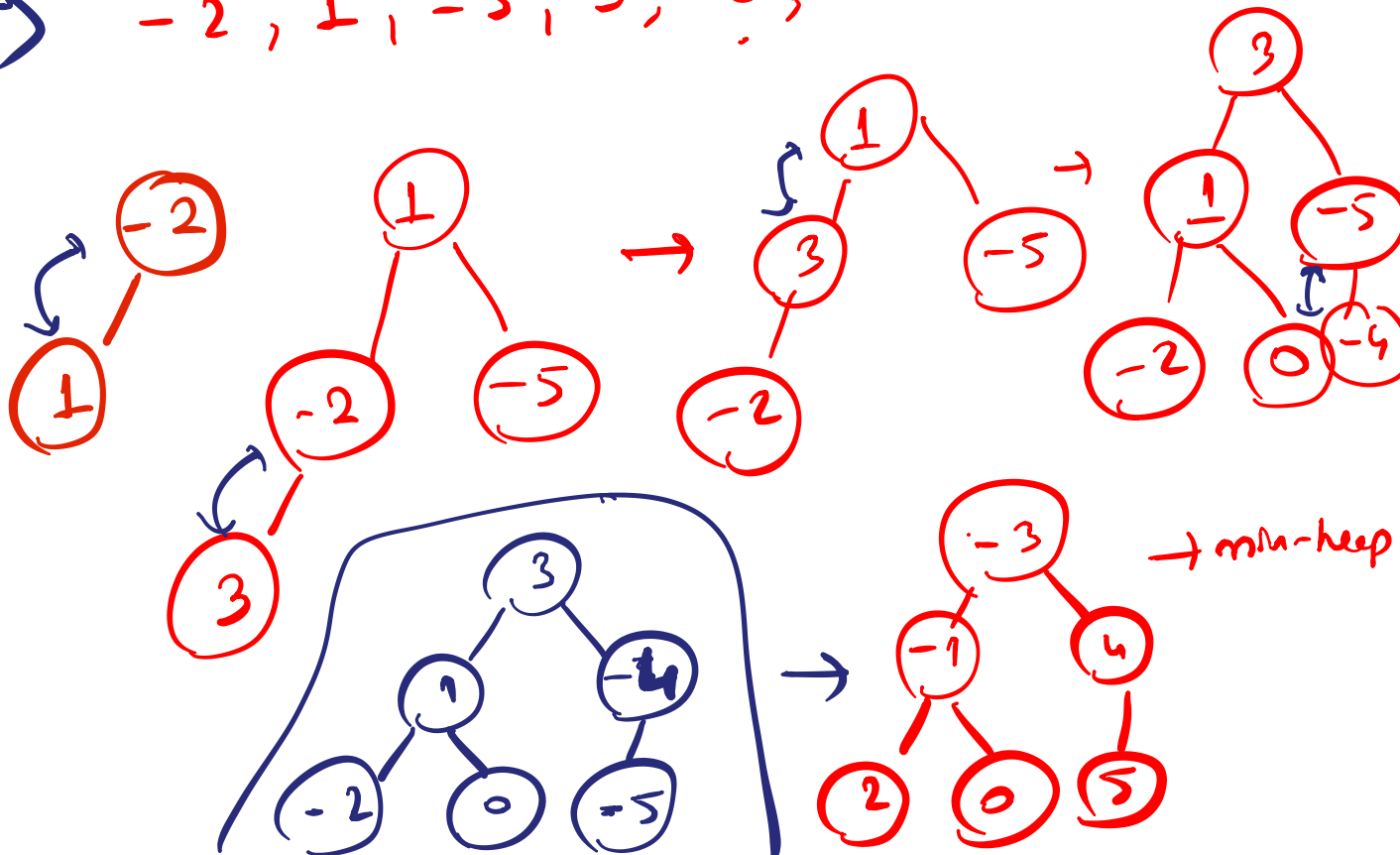
→ 2, -1, 5, -3, 0, 4

→ Replace the value K_i of a given list by $-K_i$

→ Apply a max-heap algorithm to construct a heap

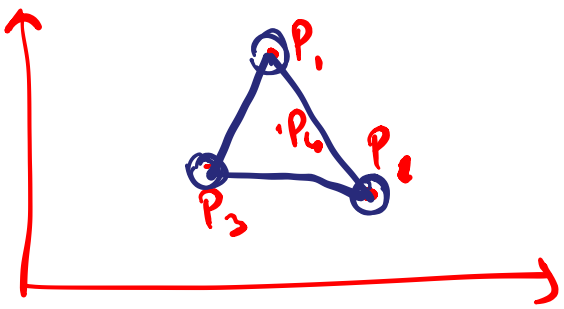
→ Change signs of the keys

→ -2, 1, -5, 3, 0, -4

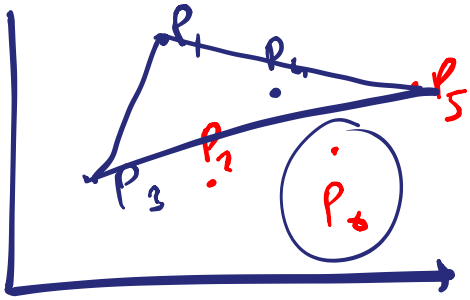


6.6

5. Given $n > 3$ points $P_1 = (x_1, y_1), \ldots, P_n = (x_n, y_n)$ in the coordinate plane, design an algorithm to check whether all the points lie within a triangle with its vertices at three of the points given. (You can either design an algorithm from scratch or reduce the problem to another one with a known algorithm.)



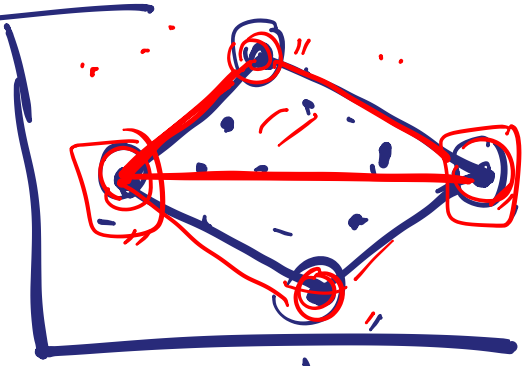
yes



No

→ Reduce the in to Convex Hull

- Apply Quic Hull





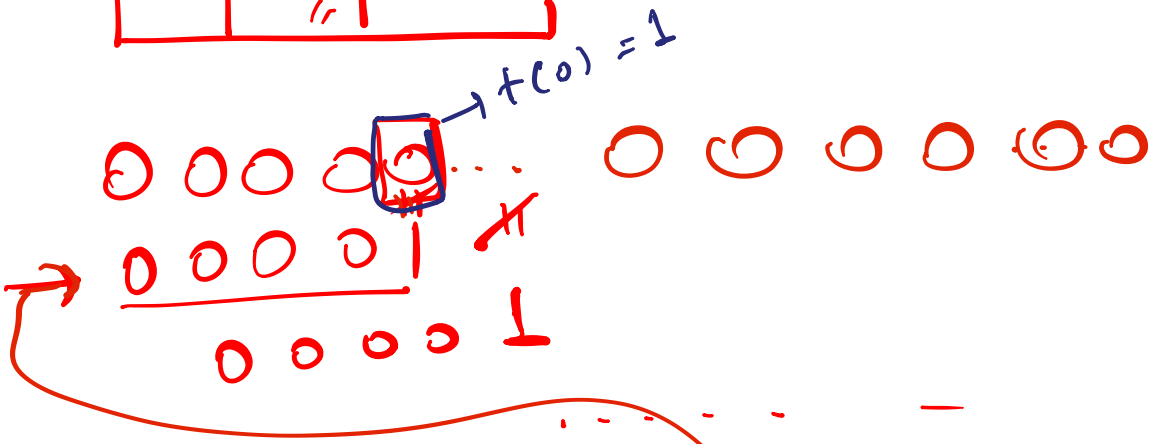
7.2

3. How many character comparisons will be made by Horspool's algorithm in searching for each of the following patterns in the binary text of 1000 zeros?

a. 00001 **b.** 10000 **c.** 01010

a)

c	0	1
t(c)	1	5



trials = $m - n + 1 = 996$

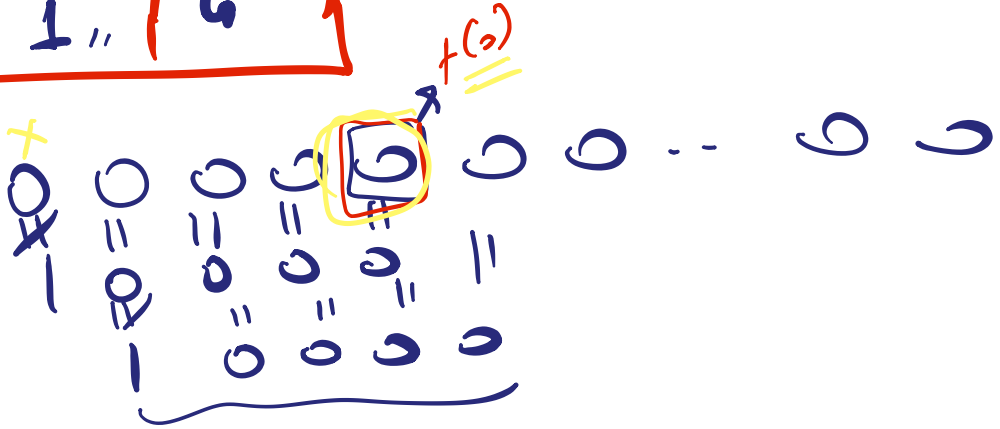
$= \left\lfloor \frac{n-n}{\text{shift increment}} \right\rfloor + 1 = \frac{1000 - 5}{1} + 1 = 996$

Comparisons = $996 \times 1 = 996$

b)

c	0	1
t(c)	1	4

pattern: 10000



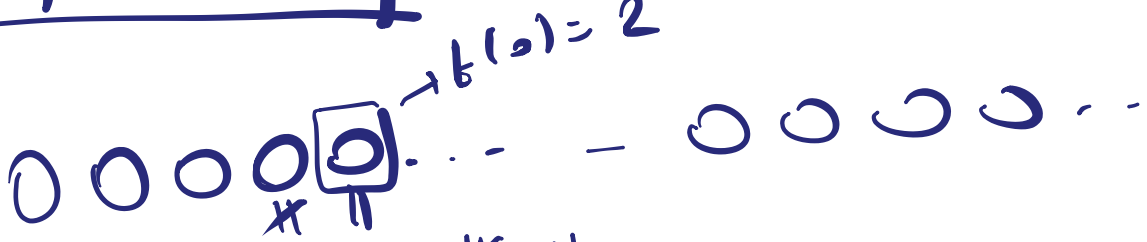
trials = $m - n + 1 = 1000 - 5 + 1 = 996$

comparisons = $996 \times 5 = 4980$

c)

c	0	1
t(c)	2	1

patterns = 0 1 0 1 0



→ 0 1 0 1 0 # 11

0 1 0 1 0

shifts

trials = $\left\lfloor \frac{1000 - 5}{2} \right\rfloor + 1 = 498$

shift amount

comparisons = $498 \times 2 = 996$

7.2

7. How many character comparisons will the Boyer-Moore algorithm make in searching for each of the following patterns in the binary text of 1000 zeros?

a. 00001 **b.** 10000 **c.** 01010

$d = \max\{d_1, d_2\}$

a)

Bad-symbol Table

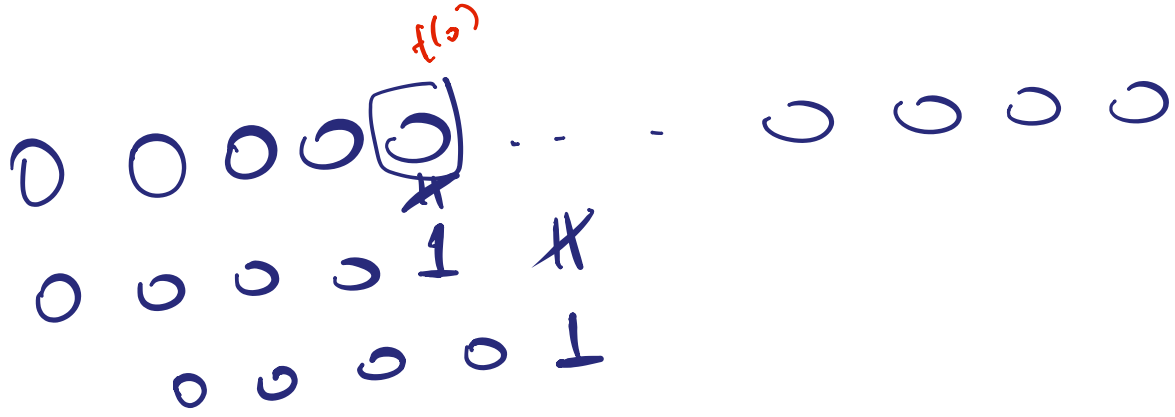
c	0	1
t(c)	1	5

(# of matching char)

$d_1 = \max\{t(c) - k, 1\}$

Good-Suffix Table

k	Pattern	d_2
1	0000**1**	5
2	000**01**	5
3	00**001**	5
4	0**0001**	5



$d_1 = \{1 - 0, 1\} = 1$, $d_2 = 0$

$d = \max\{d_1, d_2\}$

trials = $1000 - 5 + 1 = 996$

Comparisons = $996 \times 1 = 996$

b) pattern = 10002

Bad-Symbol

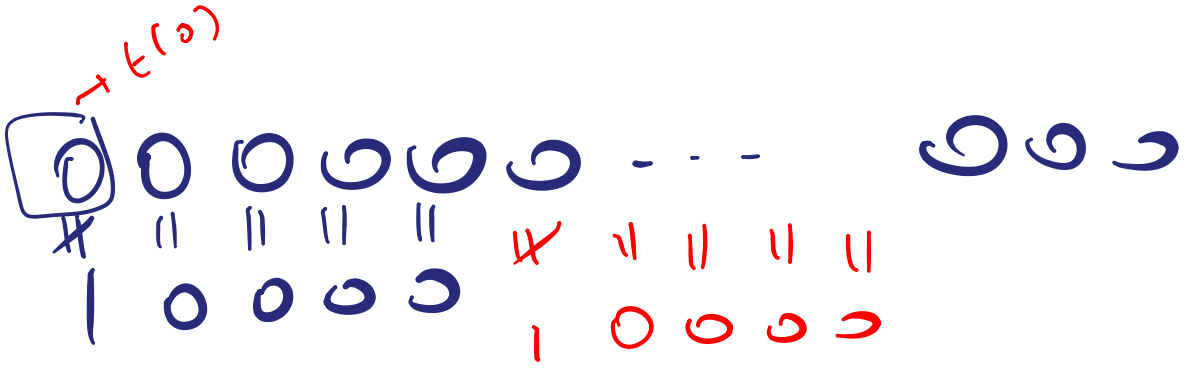
c	0	1
t(c)	1	4

Good Suffix Table

k	pattern	d_2
1	10002	3
2	10002	2
3	10002	1
4	10002	5

$d_1 = \max\{t(c) - k, 1\}$

$d = \max\{d_1, d_2\}$



$d_1 = \max\{1 - 4, 1\} = 1$

$d_2 = 5$

$d = \max\{d_1, d_2\}$

$= 5$

$\#\text{trials} = \left\lfloor \frac{1000 - 5}{5} \right\rfloor + 1 = 200$

$\#\text{comparison} = 200 \times 5 = 1000$

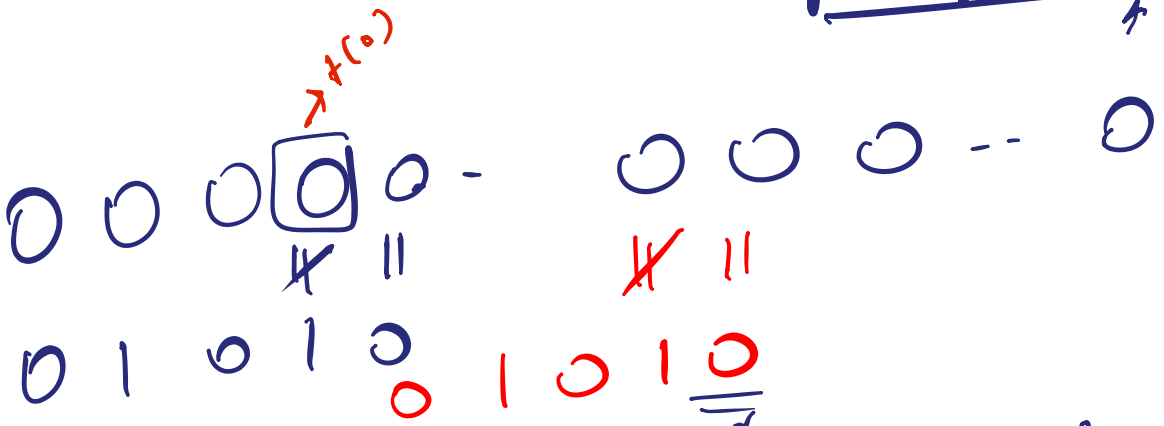
c) pattern : 0 1 0 1 0

Bad-symbol

c	0	1
t(c)	2	1

Good-Suffix

k	patterns	d_2
1	0 1 0 1 0	4
2	0 1 0 1 0	4
3	0 1 0 1 0	2
4	0 1 0 1 0	2



$d_1 = \max\{t(o) - k, 1\} = \max\{2-1, 1\} = 1$

$d_2 = 4$

$d = \max\{d_1, d_2\} = 4$

$\#\text{trials} = \left\lfloor \frac{1000-5}{4} \right\rfloor + 1 = 248$

$\#\text{comparisons} = 248 \times 2 = 496$